

Software Engineering Lab

Complete Maven → Git/GitHub → Docker → Docker Hub Practical Guide

Use this as a practical-exam command sheet. Replace YOUR_USERNAME, YOUR_REPO, YOUR_EMAIL, and other placeholders with your own values.

1. Complete Flow at a Glance

```
GitHub repository
↓
Clone / Import into Eclipse
↓
Check pom.xml
↓
Maven: mvn clean package
↓
target/*.war
↓
Git: init → add → commit → branch → remote → push
↓
Dockerfile
↓
docker build
↓
Docker Image
↓
docker run -p HOST_PORT:8080
↓
Tomcat + WAR
↓
http://localhost:HOST_PORT/app/
↓
docker tag → docker push
↓
Docker Hub
```

2. Maven — Import and Build

Eclipse import

File → Import → Git → Projects from Git (with smart import) → Clone URI → enter repository URL → select branch → choose directory → Finish.

Check Maven

```
mvn -version
```

Build the WAR

```
mvn clean package
```

Expected result: **BUILD SUCCESS**. The WAR is generated under **target/**.

Eclipse Maven build

Right-click project → Run As → Maven build... → Goals: **clean package** → Run.

Useful Maven commands

Command	Purpose
mvn -version	Check Maven and Java setup
mvn clean	Delete previous target build output

mvn package	Compile, test, and package
mvn clean package	Clean then build the WAR/JAR
mvn test	Run tests
mvn install	Build and install artifact into local .m2 repository

Important pom.xml items

```
<packaging>war</packaging>
```

```
<groupId>...</groupId>
```

```
<artifactId>...</artifactId>
```

```
<version>...</version>
```

```
<dependencies>
```

```
...
```

```
</dependencies>
```

Dependencies are downloaded to the local Maven repository, normally **.m2/repository**. Maven build output is placed in **target/**.

3. Git — Complete Start-to-End Flow

Open Git Bash at the project folder

```
cd C:/Users/YOUR_NAME/git/YOUR_PROJECT
```

Configure Git identity

```
git config --global user.name "Your Name"
git config --global user.email "YOUR_EMAIL"
git config --global user.name
git config --global user.email
```

Initialize and make first commit

```
git init
git status
git add .
git status
git commit -m "first commit"
git log --oneline
```

Create/switch to main

```
git branch
git branch -M main
git branch
```

Create your GitHub repository

Create an empty repository on GitHub, for example **SE-Lab-Practice**. Then connect your local repository:

```
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git remote -v
```

Push to GitHub

```
git push -u origin main
```

For your own repository, use your own GitHub URL. If an old remote is present, check it with **git remote -v**.

4. Git SBQ Commands

Task	Command
Status	git status
Initialize	git init
Add all	git add .
Commit	git commit -m "message"
Branches	git branch
Create branch	git branch feature
Switch branch	git switch feature
Create + switch	git switch -c feature
Merge	git switch main → git merge feature
Stash changes	git stash
See stashes	git stash list
Apply stash	git stash apply
Remove latest stash	git stash drop
Restore file	git restore filename

Revert commit	git revert COMMIT_ID
Clone	git clone URL
Pull	git pull
Push	git push
View log	git log --oneline

.gitignore example

```
target/  
.classpath  
.project  
.settings/  
*.log
```

A .gitignore tells Git which files/folders should not be tracked. For Maven projects, **target/** is commonly ignored.

5. Docker — Check Setup

```
docker --version
docker ps
docker ps -a
docker image ls
```

Start Docker Desktop first. Then run the commands above from the Maven project directory.

6. Create the Dockerfile

For a WAR deployed to Tomcat, use:

```
FROM tomcat:9.0
COPY target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

Note: 8080 is the Tomcat port inside the container. It does not have to be the port used by your Windows host.

7. Build the Docker Image

```
mvn clean package
docker build -t lmsimage .
docker image ls
```

The Maven command makes sure the WAR exists in **target/** before Docker copies it into the image.

8. Run the Docker Container

```
docker run -d -p 7089:8080 --name lmcontainer lmsimage
docker ps
```

Port mapping: **host 7089** → **container 8080**. If host port 7089 is unavailable, use another host port, for example **9098:8080**.

Open the application

```
http://localhost:7089/
```

If the WAR is named **app.war** and is deployed as app.war, Tomcat normally uses the context path **/app**, so try:

```
http://localhost:7089/app/
```

If the Dockerfile renames the WAR to **ROOT.war**, the root URL **/** is used.

9. Docker Container Management — SBQs

Task	Command
List running containers	docker ps
List all containers	docker ps -a
List images	docker image ls
Pull image	docker pull ubuntu
Run image	docker run ubuntu
Interactive mode	docker run -it ubuntu bash
Start container	docker start CONTAINER_NAME
Stop container	docker stop CONTAINER_NAME
Remove container	docker rm CONTAINER_NAME
Container logs	docker logs CONTAINER_NAME
Execute command	docker exec CONTAINER_NAME COMMAND
Build image	docker build -t IMAGE_NAME .
Tag image	docker tag IMAGE USERNAME/REPO:TAG
Push image	docker push USERNAME/REPO:TAG

10. Practice Ubuntu Interactive Mode

```
docker pull ubuntu
docker run ubuntu
docker run -it ubuntu bash
pwd
ls
exit
```

11. Docker Hub — Complete Flow

```
docker login
```

```
docker image ls
```

```
docker tag lmsimage YOUR_USERNAME/lmsimage:latest
```

```
docker image ls
```

```
docker push YOUR_USERNAME/lmsimage:latest
```

Docker Hub image naming format: **USERNAME/REPOSITORY:TAG**. Example: **aniketh123/lmsimage:latest**.

12. Full Practical Exam Sequence

```
# MAVEN
mvn -version
mvn clean package
# verify target/*.war
```

```
# GIT
git init
git config --global user.name "Your Name"
git config --global user.email "YOUR_EMAIL"
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
```

```

git push -u origin main

# DOCKER
docker --version
docker login
docker build -t lmsimage .
docker image ls
docker run -d -p 7089:8080 --name lmcontainer lmsimage
docker ps
# open http://localhost:7089/ or /app/

# DOCKER HUB
docker tag lmsimage YOUR_USERNAME/lmsimage:latest
docker push YOUR_USERNAME/lmsimage:latest

# MANAGEMENT / SBQs
docker ps -a
docker stop lmcontainer
docker start lmcontainer
docker logs lmcontainer
docker rm lmcontainer
docker pull ubuntu
docker run -it ubuntu bash

```

13. Troubleshooting Quick Reference

Problem	What to check
Java source/target error	Check maven-compiler-plugin in pom.xml.
Old WAR plugin error	Use a modern maven-war-plugin version compatible with your environment.
No WAR in target	Run mvn clean package and look for BUILD SUCCESS.
Docker build cannot find WAR	Make sure target/*.war exists before docker build.
Tomcat opens but 404	Check WAR name/context path and docker logs CONTAINER_NAME.
Port already in use	Change only the host side: -p 9098:8080.
Container stopped	Run docker ps -a and docker logs CONTAINER_NAME.

Exam memory trick: Maven creates the WAR. Git stores/version-controls and pushes the project. Docker packages/runs the WAR. Docker Hub stores the Docker image.